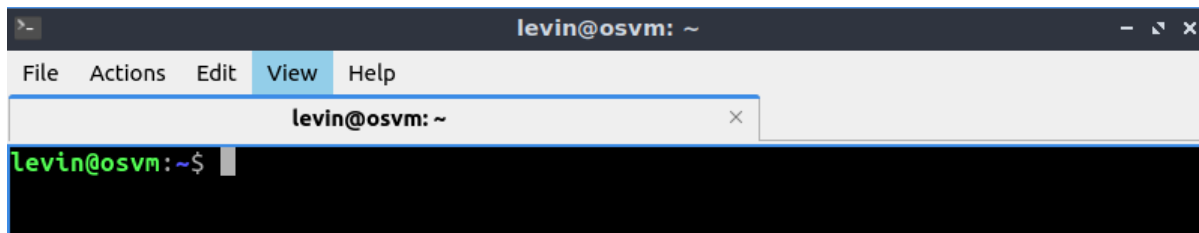
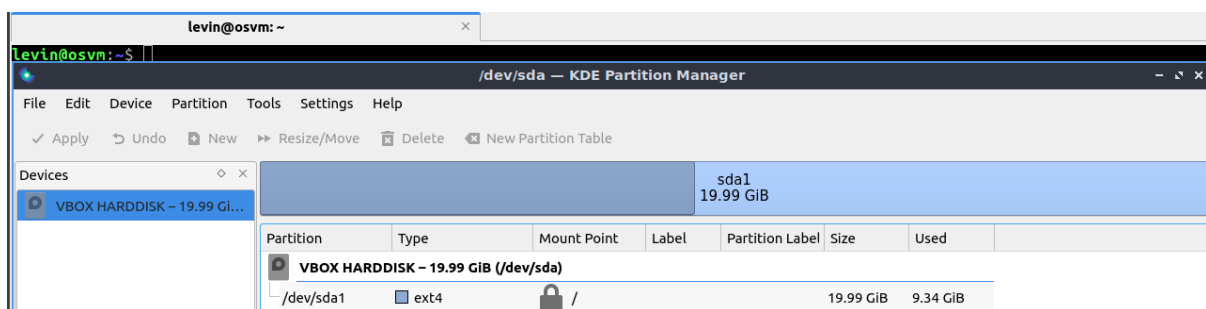


Task 1: Installing Linux OS

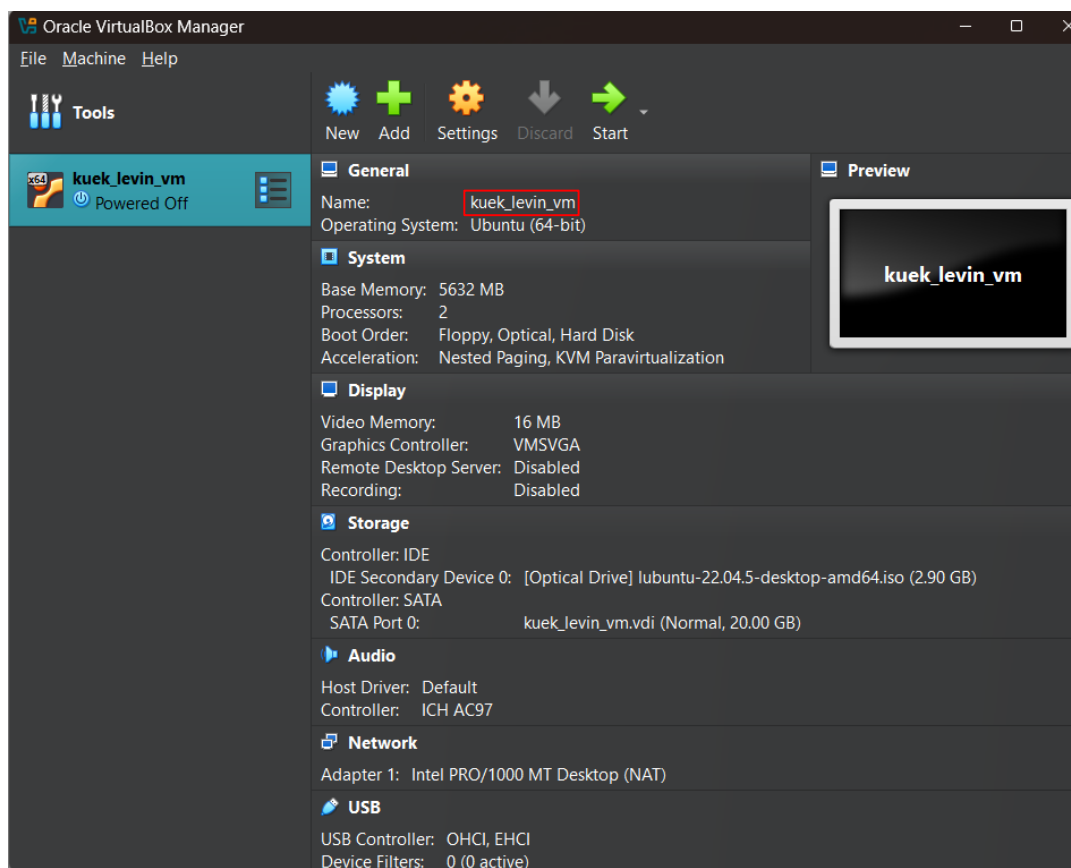
1. Screenshot of terminal.



2. The partition information.



3. The spec of the virtual machine (VM).



Task 2: CPU Resource Monitoring Program

When creating the shell script, only the owner can read and write, but no one can execute it.

Thus, the command `chmod a+x CPU.sh` to allow everyone to execute the shell script.

Content of CPU.sh

```
levin@osvm: ~
#!/bin/bash
#date: 19042025
#purpose: monitor CPU resources

#-----START-----

# g) Check user got provide period monitoring or not (default: 60s)
sec=${1:-60}

echo "Period of monitoring: ${sec} seconds"
echo "Starting to monitor...."

# a) Clear previous content in csv file
>CPU.csv

# f) Non-stop monitoring
while true; do

    # b) Current time
    cur_time=$(date +"%H:%M:%S")

    # c) and d) Find highest CPU usage process
    high_cpu_proc=$(ps -eo %cpu,comm --sort=-%cpu |head -n 2 | tail -n 1)

    # Put into separate variables
    cpu=$(echo "$high_cpu_proc" | awk '{print $1}')
    proc_name=$(echo "$high_cpu_proc" | awk '{print $2}')

    # e) Write in csv file
    echo "$cur_time ; $proc_name ; $cpu" >> CPU.csv

    # g) Run sleep
    sleep $sec

done

#-----END-----
```

Content of CPU.csv

```
levin@osvm: ~  
levin@osvm:~$ cat CPU.csv  
21:17:02 ; Isolated ; 10.5  
21:17:08 ; Isolated ; 10.5  
21:17:14 ; Isolated ; 10.4  
21:17:20 ; Isolated ; 10.4  
21:17:26 ; Isolated ; 10.5  
21:17:32 ; Isolated ; 10.5  
21:17:38 ; Isolated ; 10.5  
21:17:44 ; Isolated ; 10.5  
21:17:50 ; soffice.bin ; 15.6  
21:17:56 ; Isolated ; 10.5  
21:18:02 ; screengrab ; 12.0  
21:18:08 ; Isolated ; 10.4  
21:18:14 ; Isolated ; 10.4
```

The command “cat CPU.csv” is used to display the content that has recorded by the CPU.sh.

Run CPU.sh in background of terminal and perform verification

```
levin@osvm: ~  
levin@osvm:~$ ./CPU.sh 4 &  
[1] 3198  
levin@osvm:~$ Period of monitoring: 4 seconds  
Starting to monitor.....  
ps -all  
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD  
0 S  1000    3198    3188  0  80   0 - 2527 do_wai pts/0        00:00:00 CPU.sh  
0 S  1000    3240    3198  0  80   0 - 2094 hrtime pts/0        00:00:00 sleep  
4 R  1000    3241    3188  0  80   0 - 3173 -      pts/0        00:00:00 ps
```

Content of CPU-stop.sh

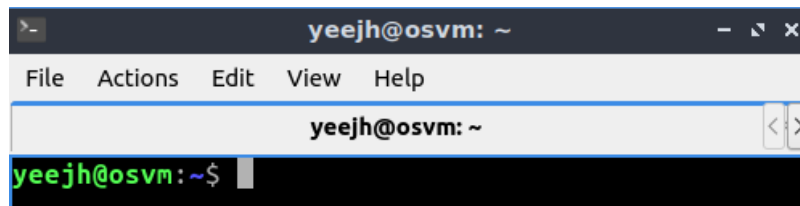
```
levin@osvm: ~  
#!/bin/bash  
#date: 19042025  
#purpose: kill CPU.sh task if running  
  
#-----START-----  
  
# Find the PID of CPU.sh  
pid=$(pgrep -f "/bin/bash .*CPU.sh")  
  
# Check if CPU.sh exist  
if [ -z "$pid" ]; then  
    echo "CPU.sh isn't running."  
else  
    kill $pid  
    echo "CPU.sh (PID: $pid) has stopped running."  
fi  
  
#-----END-----  
~  
~  
~  
-- INSERT --
```

Verification of CPU-stop.sh

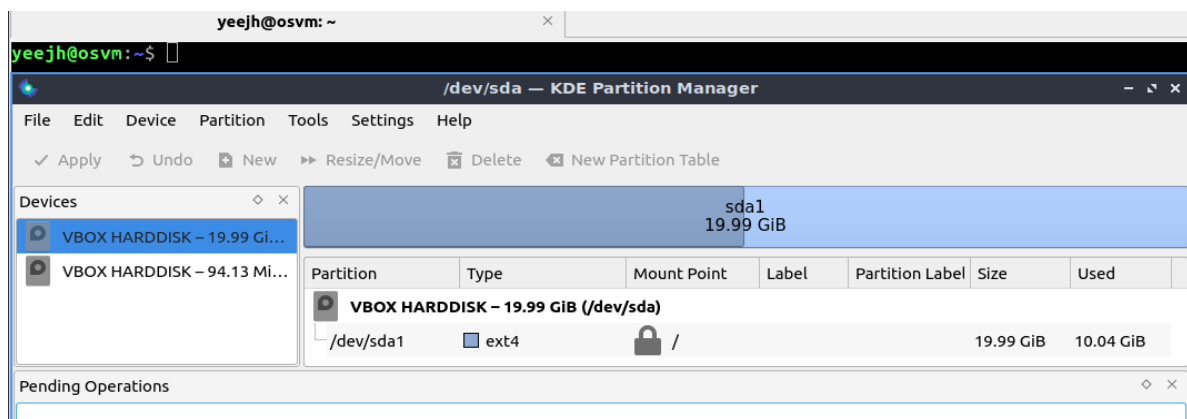
```
levin@osvm: ~  
levin@osvm:~$ ./CPU-stop.sh  
CPU.sh (PID: 3198) has stopped running.  
levin@osvm:~$ ps -all  
F S  UID      PID      PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD  
4 R  1000     3659      3587  0  80   0 -  3166 -          pts/1        00:00:00 ps  
levin@osvm:~$
```

Task 1: Installing Linux OS

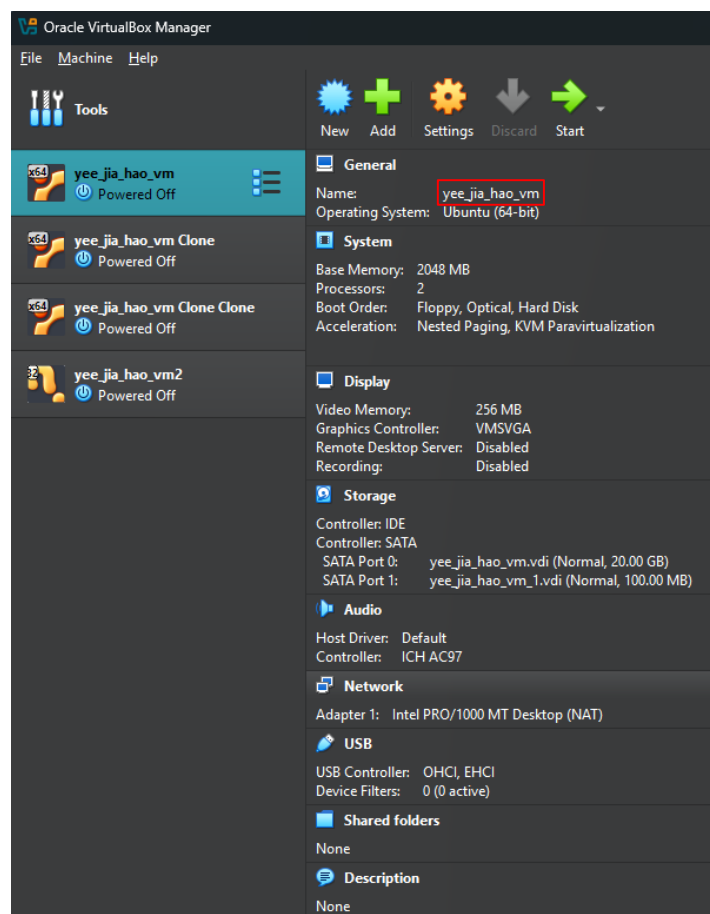
1. Screenshot of terminal.



2. The partition information.



3. The spec of the VM.



Task 2: RAM Resource Monitoring Program

When creating the shell script, only the owner can read and write, but no one can execute it.

Thus, the command `chmod a+x RAM.sh` to allow everyone to execute the shell script.

Content of RAM.sh

```
yeejh@osvm: ~  
#!/bin/bash  
#date: 18042025  
#purpose: Monitor process with highest RAM utilization.  
  
#-----START-----  
  
# Parameter 1: Sleep Period  
sec=${1:-60}  
  
echo "Period of monitoring: ${sec} seconds"  
echo "Starting to monitor....."  
  
# a) Clear previous RAM.csv content  
> RAM.csv  
  
# f) Infinite loop unless trigger (CTRL + C)  
while true; do  
    # b) Time in "hh:mm:ss" format  
    cur_time=$(date +"%H:%M:%S")  
  
    # c) and d) Get the highest RAM utilization process  
    high_mem_proc=$(ps -eo comm,%mem --sort=-%mem | head -n 2 | tail -n 1)  
  
    # e) Append process with highest mem utilization into RAM.csv  
    high_mem_proc=$(echo "$high_mem_proc" | awk '{print $1 " ; " $2}')
```

```
    echo "$cur_time ; $high_mem_proc" >> RAM.csv  
  
    # g) Period defined by user (Second) OR default 60 Seconds  
    sleep $sec  
done  
  
#-----END-----
```

Content of RAM.csv

```
yeejh@osvm:~$ cat RAM.csv  
18:46:24 ; pcmanfm-qt ; 5.1  
18:46:28 ; pcmanfm-qt ; 5.1  
18:46:32 ; firefox ; 11.3  
18:46:36 ; firefox ; 21.4  
18:46:40 ; firefox ; 20.6  
18:46:44 ; firefox ; 23.6  
18:46:48 ; firefox ; 24.9  
18:46:52 ; firefox ; 25.2  
18:46:56 ; firefox ; 25.4  
18:47:00 ; firefox ; 23.9
```

The command `cat RAM.csv` is used to display the content that has recorded by the RAM.sh script.

Run RAM.sh in background of terminal and perform verification

```
yeejh@osvm:~$ ./RAM.sh 4 &
[1] 3702
yeejh@osvm:~$ Period of monitoring: 4 seconds
Starting to monitor.....
ps -all
F S  UID      PID      PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S   1000      3702      3695  0   80   0  -  2527 do_wai pts/0        00:00:00 RAM.sh
0 S   1000      5162      3702  0   80   0  -  2094 hrtime pts/0        00:00:00 sleep
0 R   1000      5164      3695  0   80   0  -  3169 -      pts/0        00:00:00 ps
```

Content of RAM-stop.sh

```
yeejh@osvm: ~
#!/bin/bash
#purpose: Check and Kill all RAM.sh process running under /bin/bash.

#-----START-----

# Only find the PID of Running RAM.sh in /bin/bash
PID=$(pgrep -f "/bin/bash .*RAM.sh")

# Kill executing RAM.sh if exist.
if [ -n "$PID" ]; then
    kill $PID
    echo "Successfully Kill RAM.sh (PID: $PID)"
else
    echo "RAM.sh is not running!"
fi
# End of Conditional Block

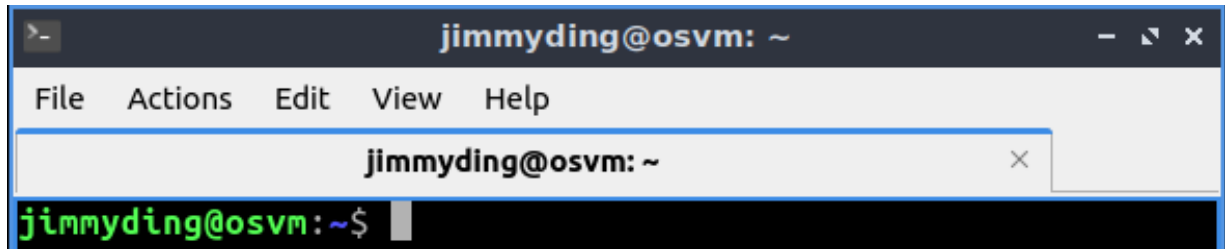
#-----END-----
```

Verification of RAM-stop.sh

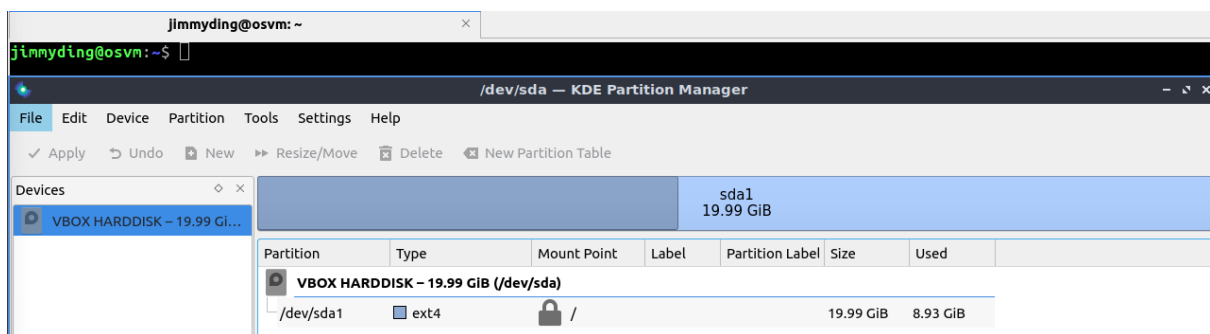
```
yeejh@osvm:~$ ./RAM-stop.sh
Successfully Kill RAM.sh (PID: 3702)
[1]+  Terminated                  ./RAM.sh 4
yeejh@osvm:~$ ps -all
F S  UID      PID      PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 R   1000      5187      3695  0   80   0  -  3162 -      pts/0        00:00:00 ps
yeejh@osvm:~$
```

Task 1: Installing Linux OS

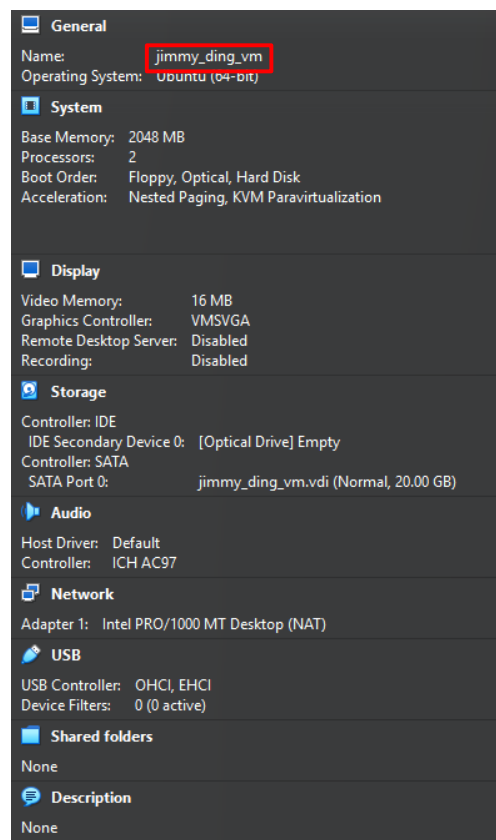
1. Screenshot of terminal.



2. The partition information.



3. The specification of the VM.



Task 2: Virtual Memory Resource Monitoring Program

When creating the shell script, only the owner can read and write, but no one can execute it.

Thus, the command `chmod a+x VM.sh` to allow everyone to execute the shell script.

Content of VM.sh

```
jimmyding@osvm: ~  
#!/bin/bash  
#purpose: Monitor Virtual Memory Resources  
  
#START  
  
# a) Clear previous content in the VM.csv  
>VM.csv  
  
# g) Allow user to provide monitoring period with 60 seconds as the default  
second=${1:-60}  
echo "Period: ${second} second"  
  
# f) Continuous monitoring  
while true;  
do  
    # b) Current date  
    current_time=$(date +%H:%M:%S)  
  
    # c) and d) Find highest CPU usage process  
    highest_memory=$(ps -eo comm,vsz --sort=-vsz | head -n 2 | tail -n 1)  
  
    # e) Replacing the space with ; and appending it to the csv file  
    highest_memory=$(echo "$highest_memory" | sed 's/ \+;/g')  
    echo "$current_time ; $highest_memory" >> VM.csv  
  
    # g) Sleep  
    sleep $second  
done
```

Content of VM.csv

```
jimmyding@osvm:~$ cat VM.csv  
22:34:14 ; pcmanfm-qt; 1345952  
22:34:18 ; pcmanfm-qt; 1345952  
22:34:22 ; pcmanfm-qt; 1345952  
22:34:26 ; firefox; 11137224  
22:34:30 ; firefox; 11240048  
jimmyding@osvm:~$
```

The '`cat VM.csv`' command is used to view the CSV that was recorded with `VM.sh`.

Run VM.sh in background of terminal and perform verification

```
jimmyding@osvm:~$ ./VM.sh 4 &
[1] 1343
jimmyding@osvm:~$ Period: 4 second
ps -all
F S  UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S   1000     1343     1336  0   80   0  -  2494 do_wai pts/0        00:00:00 VM.sh
0 S   1000     1353     1343  0   80   0  -  2094 hrtime pts/0        00:00:00 sleep
0 R   1000     1355     1336  0   80   0  -  3169 -      pts/0        00:00:00 ps
jimmyding@osvm:~$
```

Content of VM-stop.sh

```
jimmyding@osvm: ~
#!/bin/bash
#Purpose: kill VM.sh if it's running

# Find the process ID of VM.sh
process=$(pgrep -f "/bin/bash .*VM.sh")

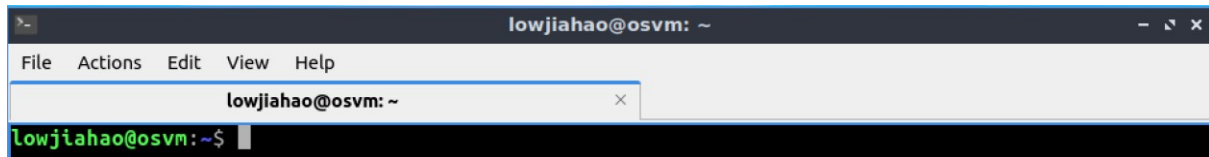
# If VM.sh is running, kill it
if [ -n "$process" ]; then
    kill $process
    echo "Successfully terminated VM.sh! (PID: $PID)"
else
    echo "VM.sh is not running!"
fi
```

Verification of VM-stop.sh

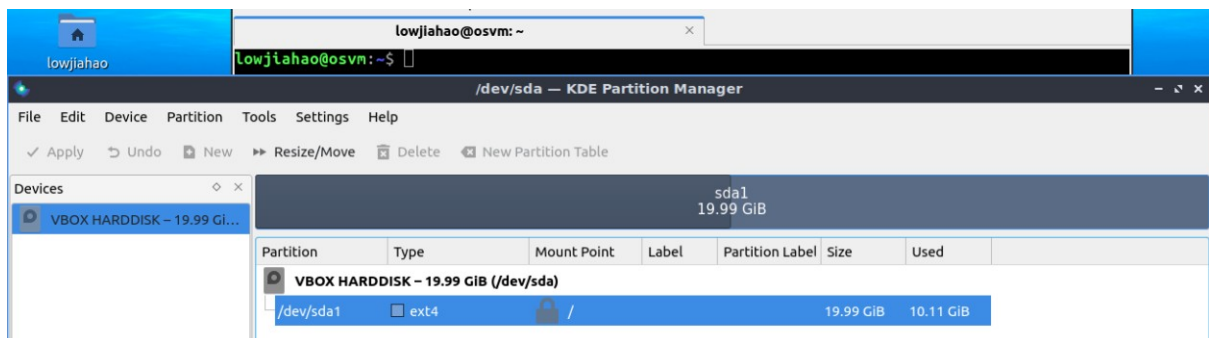
```
jimmyding@osvm:~$ ./VM-stop.sh
Successfully terminated VM.sh! (PID: 1343)
[1]+  Terminated                  ./VM.sh 4
jimmyding@osvm:~$ ps -all
F S  UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 R   1000     1398     1336  0   80   0  -  3162 -      pts/0        00:00:00 ps
jimmyding@osvm:~$
```

Task 1: Installing Linux OS

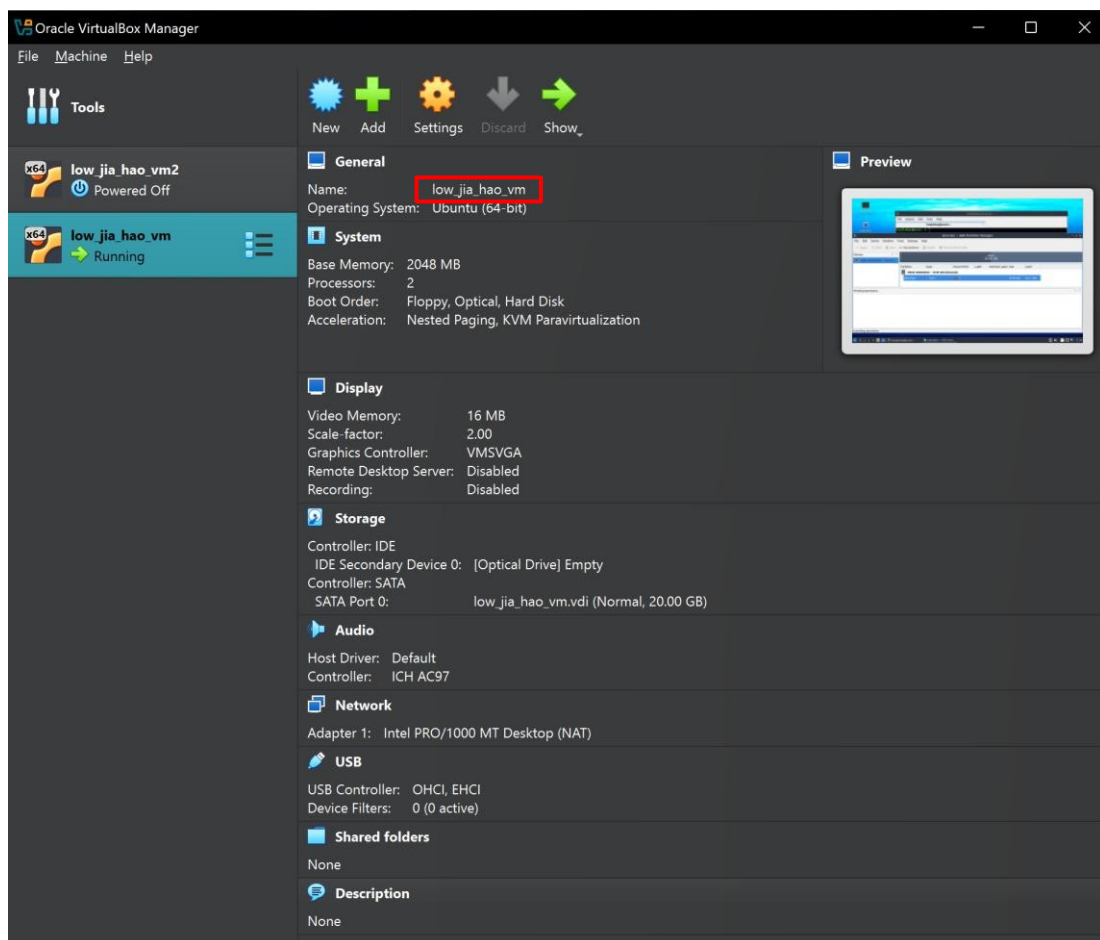
1. Screenshot of terminal



2. The partition information



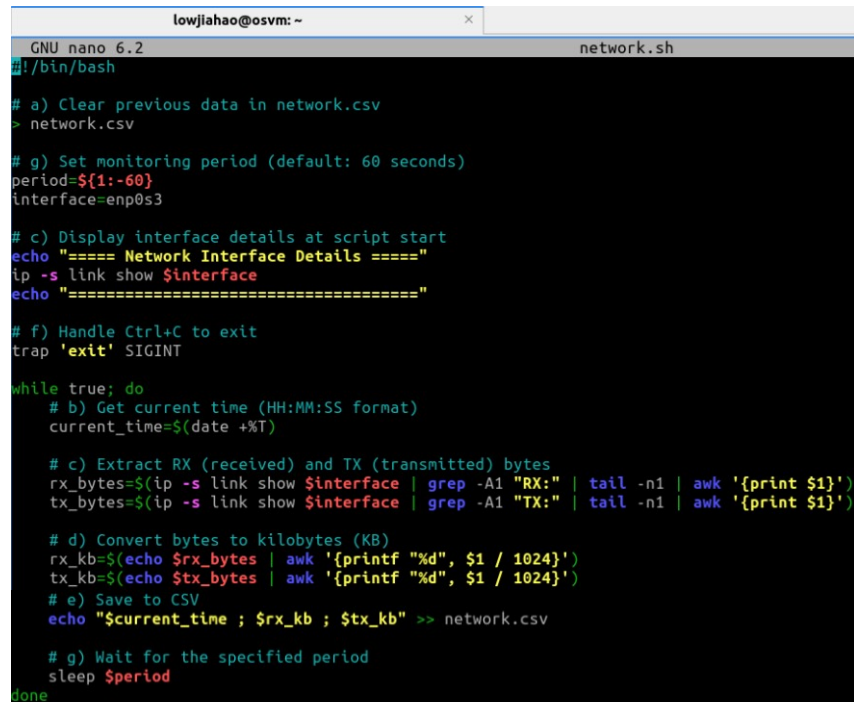
3. The spec of the VM



Task 2: Network Usage Monitoring Program

When creating the shell script, it only can read and write, it was unable to be executed. Thus, the command “`chmod +x network.sh`” to allow the shell script to be executable.

Content of network.sh



```
lowjiahao@osvm: ~
GNU nano 6.2 network.sh
#!/bin/bash

# a) Clear previous data in network.csv
> network.csv

# g) Set monitoring period (default: 60 seconds)
period=${1:-60}
interface=enp0s3

# c) Display interface details at script start
echo "==== Network Interface Details ====="
ip -s link show $interface
echo "===== "

# f) Handle Ctrl+C to exit
trap 'exit' SIGINT

while true; do
    # b) Get current time (HH:MM:SS format)
    current_time=$(date +%T)

    # c) Extract RX (received) and TX (transmitted) bytes
    rx_bytes=$(ip -s link show $interface | grep -A1 "RX:" | tail -n1 | awk '{print $1}')
    tx_bytes=$(ip -s link show $interface | grep -A1 "TX:" | tail -n1 | awk '{print $1}')

    # d) Convert bytes to kilobytes (KB)
    rx_kb=$(echo $rx_bytes | awk '{printf "%d", $1 / 1024}')
    tx_kb=$(echo $tx_bytes | awk '{printf "%d", $1 / 1024}')

    # e) Save to CSV
    echo "$current_time ; $rx_kb ; $tx_kb" >> network.csv

    # g) Wait for the specified period
    sleep $period
done
```

The command “`./network.sh 4`” is used to run network.sh script with 4 second as the period.

Content of network.csv

```
lowjiahao@osvm:~$ cat network.csv
18:48:52 ; 43493 ; 2517
18:48:56 ; 43493 ; 2517
18:49:00 ; 43493 ; 2517
18:49:04 ; 43493 ; 2517
18:49:08 ; 43493 ; 2517
18:49:12 ; 43493 ; 2517
18:49:16 ; 43494 ; 2518
18:49:20 ; 43494 ; 2518
18:49:24 ; 43494 ; 2518
18:49:29 ; 43494 ; 2518
18:49:33 ; 43494 ; 2518
18:49:37 ; 43494 ; 2518
18:49:41 ; 43494 ; 2518
18:49:45 ; 43494 ; 2518
18:49:49 ; 43495 ; 2518
18:49:53 ; 43495 ; 2519
18:49:57 ; 43495 ; 2519
18:50:01 ; 43495 ; 2519
18:50:05 ; 43495 ; 2519
18:50:09 ; 43495 ; 2519
18:50:13 ; 43495 ; 2519
18:50:17 ; 43495 ; 2519
18:50:21 ; 43495 ; 2519
18:50:25 ; 43495 ; 2519
18:50:29 ; 43495 ; 2519
18:50:33 ; 43496 ; 2520
18:50:37 ; 43496 ; 2520
18:50:41 ; 43508 ; 2529
18:50:45 ; 43755 ; 2607
18:50:49 ; 43999 ; 2658
18:50:53 ; 44354 ; 2727
18:50:57 ; 44389 ; 2776
18:51:02 ; 44389 ; 2780
18:51:06 ; 44396 ; 2786
18:51:10 ; 44406 ; 2792
18:51:14 ; 44422 ; 2808
18:51:18 ; 44428 ; 2811
18:51:22 ; 44429 ; 2811
18:51:26 ; 44430 ; 2813
lowjiahao@osvm:~$
```

The command “cat network.csv” is used to display the content that has recorded by the network.sh script.

Run network.sh in background of terminal and perform verification

```
lowjiahao@osvm:~$ ./network.sh 4 &
[1] 1276
lowjiahao@osvm:~$ ===== Network Interface Details =====
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 08:00:27:e0:fa:9d brd ff:ff:ff:ff:ff:ff
    RX:  bytes  packets  errors  dropped  missed  mcast
         1538      10       0       0       0       1
    TX:  bytes  packets  errors  dropped  carrier  collsns
         9247      69       0       0       0       0
=====
lowjiahao@osvm:~$ ps -all
F S  UID      PID    PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S  1000    1276     1079  0  80   0 -  2527 do_wai pts/0        00:00:00 network.sh
0 R  1000    1313     1276  0  80   0 -    0 - pts/0        00:00:00 sleep
0 R  1000    1314     1079  0  80   0 -  3169 - pts/0        00:00:00 ps
lowjiahao@osvm:~$
```

Content of network-stop.sh

```
lowjiahao@osvm: ~  
GNU nano 6.2 network-stop.sh  
#!/bin/bash  
# Purpose: Find and Kill network.sh process  
  
# Find the process ID of network.sh  
process=$(pgrep -f "^/bin/bash .*network.sh")  
  
# If network.sh has been found, kill it  
if [ -n "$process" ]; then  
    kill $process  
    echo "network.sh has been found and terminated at (PID: $process)"  
else  
    echo "network.sh is not running."  
fi
```

Verification of network-stop.sh

```
lowjiahao@osvm:~$ ./network-stop.sh  
network.sh has been found and terminated at (PID: 1276)  
[1]+  Terminated                  ./network.sh 4  
lowjiahao@osvm:~$ ps -all  
F S  UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD  
0 R  1000     1641     1079  0   80   0 -  3162 -      pts/0    00:00:00 ps  
lowjiahao@osvm:~$
```